

Training a 0.6B C++ Code LLM from Scratch • Stage 1

Pretraining Until It Writes Basic C++

Hand-written Transformer + from-scratch pretraining • The Stack v2 C++ • Full pipeline (data/tokenizer/model/training) + first-principles derivation of CE / perplexity

Outline (Stage A: pretrain → writes C++)

- **0.** Goal & full pipeline: pretrain → SFT → RL; this piece does only step A
- **1.** The "prose-writing → problem-solving" paradigm: the essence of next-token
- **2.** Data: The Stack v2 C++ cleaning (dedup / quality / license / PII)
- **3.** Tokenizer: BPE + C++ keyword protection tokens
- **4.** From ID to embedding: one-hot, table lookup, end-to-end learned representation
- **5.** Model: 0.6B decoder (RoPE / SwiGLU / RMSNorm / 32 layers deep)
- **6.** Training mechanics: CE loss, BP updating embeddings, gradient accumulation
- **7.** First-principles of the smoke test: why initial loss = $\log|\text{vocab}|$, ppl = $|\text{vocab}|$
- **8.** The intuition of perplexity: "how many words you're effectively choosing among"
- **9.** Memory back-and-forth: activations, grad checkpointing, OOM trade-offs
- **10.** Speed back-and-forth: GPU utilization, async prefetch, locating the real bottleneck
- **11.** Result: the steep-drop and plateau of the loss curve

0. Goal & full pipeline

Goal: train a **0.6B decoder-only Transformer** from scratch (random init) to **write C++** — not by calling an existing model, but building data cleaning, tokenizer, model, and training loop entirely by hand, walking the full path "how one trains a code LLM" once.

A. Pretrain (this piece): next-token on massive C++ code → learns to "write C++".

B. SFT: fine-tune on problem-solution data → align to "read a problem, write a solution".

C. RL (RLVR): reward = whether tests pass → force "actually gets AC".

Model size chosen by compute: 0.6B (trainable on a single A100); data is the C++ subset of The Stack v2, ~14B tokens after cleaning — right around the Chinchilla-optimal point for 0.6B.

1. From "prose-writing" to "problem-solving": the essence of next-token

Pretraining's core is **next-token prediction**: given the preceding tokens, predict the next. Its essence is language modeling — learning "what token sequences are natural, matching corpus statistics".

Key distinction: next-token learns "looks right" (language statistics), not "is correct" (logical correctness). "1+1=3" is a grammatically fine string; next-token sees nothing wrong. So **pretraining can learn to "write legal, plausible C++" (syntax, structure, common patterns), but does not guarantee "solving problems"** — that needs later SFT/RL. This piece stops at "writes C++".

For code, next-token must learn **structural dependencies** between tokens: bracket matching, variable define-use, scope, control flow. These are "long-range precise dependencies" — exactly what attention excels at, which is why Transformers suit code so well.

2. Data: cleaning The Stack v2 C++

Use the C++ subset of `the-stack-dedup` (near-dedup already done officially, saving the heaviest MinHash step). Stream-download + lightweight quality filtering, write jsonl shards directly.

Cleaning rules (use the dataset's built-in quality fields — more accurate than heuristics)

- **File size:** $200 \text{ chars} < n < 1\text{M}$ (too small = no content, too large = exceeds context).
- **Long lines:** drop if `max_line_length > 1000` (minified code or data blobs).
- **Avg line length:** drop if `avg_line_length > 200`.
- **Alphanumeric fraction:** drop if `alphanum_fraction < 0.25` (base64/binary).
- **License:** keep only permissive (MIT/Apache/BSD...).
- **Autogenerated:** drop if header contains "do not edit"/"autogenerated".

88% retention: because the-stack is pre-cleaned (few junk files) + the dedup version is already deduplicated. Final: 5.6M files → **full traversal of all data (6.4M raw files, 100% coverage)**.

Tokenize into token shards (.bin)

Each C++ file is tokenized to an ID sequence, files separated by `<eos>`, concatenated into one long token stream, split into `.bin` shards of 200M tokens each (uint16 — enough for vocab<65536). Training mmmaps the shards, zero-copy.

Engineering detail: single `tok.encode()` uses only 1 core (250 docs/s); switching to `tok.encode_batch()` multi-threads across 32 cores → 1000+ docs/s, cutting 6 hours to 1.5. **A live example of Amdahl's law — parallelism only speeds up the encode step; reading files/json/writing disk stay single-threaded, so it's 4-5× not 32×.**

Final: 5.6M docs → 13.99B tokens, 70 shards (~28GB).

3. Tokenizer: BPE + C++ keyword protection

The base is the same as NLP: byte-level BPE (subword splitting, no OOV for identifiers/literals). But code has a quirk — **keywords and symbols should be whole tokens, not chopped oddly by BPE** (e.g. `int` shouldn't split into `in|t`).

Approach (hybrid): train a BPE vocab + force-add C++ keywords (`int/for/class/...`) and multi-char symbols (`::/->/<=>/...`) as **protected tokens**, forbidden from being split.

- Unbounded things (identifiers, literals) → covered by BPE subwords (no OOV).
- Bounded fixed things (keywords, symbols) → whole tokens (clean).

This resolves both extremes: "pure lexer's vocab explosion" and "pure BPE mangling keywords".

The training-size trap: BPE doesn't need the full 14B — feeding too much initially ran 139 minutes without finishing. Following StarCoder2's approach (train on a "small subset" of The Stack), **sampling 50k files (a few hundred M tokens) trains in 40 seconds** — single language + vocab 32K is plenty. Final vocab = 32006 (32000 BPE + special/protected tokens).

4. From ID to embedding: one-hot, lookup, end-to-end learning

Tokenization produces a **token ID sequence** (each token's index in the vocab, 0–32005). The model eats IDs, not text.

An ID is the compact form of one-hot

Token ID = 856 is equivalent to a 32006-dim one-hot (1 at position 856, 0 elsewhere). **Vocab size 32006 = one-hot dimension.** But storage/compute uses the integer ID (space-saving), not the expanded sparse one-hot.

embedding = one-hot × embedding matrix = taking row 856 of the matrix. So

`nn.Embedding` just "indexes the row by ID", skipping the sparse multiply (equivalent but fast).

Embeddings are learned (not pre-existing)

The embedding table ($32006 \times d_{\text{model}}$) is randomly initialized and **learned end-to-end with the next-token task** — not word2vec-style pretrained-and-frozen word vectors. CE's gradient flows back through BP to the embedding table, updating each (seen) token's vector.

Idea correspondence: this mirrors REALM's "retriever representation learned end-to-end with the downstream task" — representation isn't fixed independently, but shaped by the final task. Two-tower embeddings in ranking, shaped by the ranking task, follow the same principle.

5. Model: the 0.6B decoder

Component	Choice	Note
Size	$d=1280$, $L=32$, heads=20	675M; deep (32 layers) aids multi-step reasoning
Norm	RMSNorm	simpler & more stable than LayerNorm
Position	RoPE	parameter-free; context change doesn't add params
FFN	SwiGLU	$\text{ffn_dim} \approx 2.67 \times d$, modern standard
Attention	SDPA (FlashAttention)	doesn't store the $n \times n$ matrix, saves memory
Other	tie_embeddings, residual scaled init	save params, stabilize deep training

Params vs activations vs input length (the three relationships): params ($W_q/W_k/W_v/\text{FFN}$) are determined only by d_{model} , **independent of seq length** (all tokens share one set of params — reuse). Longer seq only grows activations (the $n \times n$ attention matrix, per-token intermediates), not params. That "32 layers deep" trains at all relies on **residual connections** (each layer $x + f(x)$), gradient flows straight through).

6. Training mechanics: CE, BP, gradient accumulation

Forward → CE → backward

The model outputs a 32006-dim distribution (softmax) per position. CE loss computes, per position, the "−log probability of the correct token", then averages over all positions. BP propagates gradients back via the chain rule, updating all params — including the embedding table (only rows of tokens appearing in the batch, sparse update).

micro-batch vs effective batch (gradient accumulation)

micro_batch = number of sequences fed to the GPU in one forward (memory-bound; too large → OOM).

effective batch = total amount that actually updates params once (needs to be large for stability, e.g. 0.5M tokens/step).

Memory can't fit a large batch → **gradient accumulation**: accumulate gradients over N micro-batches before one update.

$\text{eff_batch} = \text{micro_batch} \times \text{seq} \times \text{grad_accum}$. Tuning micro_batch (to avoid OOM) doesn't change eff_batch (grad_accum auto-compensates) — it only affects memory and speed, not training outcome.

step vs epoch: 1 step = one effective batch (0.49M tokens) updating once = 1/28000 of the data. Training the full 14B data once (1 epoch) = 14B / 0.49M = **28000 steps**. So total_steps=28000 = one full pass.

7. First-principles of the smoke test: initial loss = $\log|\text{vocab}|$

The first thing at training launch: check whether step-0 loss $\approx \log|\text{vocab}|$ — the standard probe verifying "correct init, no bug". Derivation:

$$L = -(1/N) \sum_i \sum_c y_{\{i,c\}} \cdot \log \hat{y}_{\{i,c\}}$$

Two ingredients:

- **Label y is one-hot**: at each position only the correct token's dim is 1, rest 0 → the inner sum over c **collapses** to just the correct-token term $-\log \hat{y}_{\{i, \text{correct}\}}$.
- **Random init → prediction $\hat{y} \approx \text{uniform}$** : random weights (small std) → logits ≈ 0 → softmax $\approx \text{uniform}$ → each dim $\approx 1/|\text{vocab}|$.

Substituting (uniform → correct-token prob = $1/|\text{vocab}|$, same at every position, unchanged after averaging):

$$L = -(1/N) \sum_i \log(1/|\text{vocab}|) = \log|\text{vocab}|$$

Empirical check: vocab=32006 → $\log(32006) \approx 10.37$. Training step-0 loss = **10.63** ≈ 10.37 ✓. The model is correctly initialized (uniform guessing), no bug, starting from the random baseline. If much larger → init problem; if much smaller → data leak / label misalignment.

8. Perplexity: "how many words you're choosing among"

Perplexity = e^{CE} . From § 7, at random init:

$$\text{ppl} = e^L = e^{\{\log|\text{vocab}|\}} = |\text{vocab}|$$

The deeper intuition — let the number of words in the uniform distribution be k :

$$\text{uniform over } k \text{ words} \rightarrow \text{CE} = \log k \rightarrow \text{ppl} = e^{\{\log k\}} = k$$

So ppl = "the model's uncertainty, equivalent to uniformly hesitating among how many words" (effective candidate count).

- Random init: $\text{ppl} = |\text{vocab}| = 32006$ (guessing among all 32k words).
- After training $\text{ppl} = 4.8$: uncertainty equivalent to "choosing among ~5 words" — quite confident.
- **Lower is better**: shrinking from "32k candidates" to "5 candidates" = the model learned C++'s strong regularities.

Code's ppl is naturally low (single digits) — syntax is highly regular, far more predictable than natural language (tens).

9. Memory back-and-forth: activations, grad checkpointing, OOM

This stretch is a process of repeated trial-and-error worth recording — the core conflict: **the activations of large context + deep model are huge, conflicting with "turn off grad_ckpt for speed"**.

OOM repeatedly (with grad_ckpt off)

Config (grad_ckpt off)	Memory	Result
8K context, micro16	78.5G	OOM
8K, micro8	79.2G	OOM
4K, micro12	79.1G	OOM
4K, micro8	78.6G	OOM

Misjudgment + diagnosis: first assumed lowering micro_batch would fix it; the more I lowered and tried, the more it OOM'd. The real cause — **with grad_ckpt off, all 32 layers' activations are stored; the base is too large (60-70G), so any micro_batch OOMs**. SwiGLU's intermediate dim 3456 means one layer's activations are ~1G, 32 layers ~38G, plus attention/temporaries/fragmentation → approaching 80G.

Precise activation-memory estimate

Per-layer activations $\approx b \times S \times (4 \times d + 3 \times \text{ffn} + 2 \times d) \times 2$ bytes. $b=8$, $S=4096$, $d=1280$, $\text{ffn}=3456$: per layer $\approx 8 \times 4096 \times 18048 \times 2 \approx 1.18\text{G} \rightarrow 32$ layers $\approx$ **38G**.

+ fixed overhead (params 1.35 + grads 1.35 + Adam-FP32 5.4 $\approx$ 8G) + temporaries + PyTorch caching/fragmentation ($\times 1.5$ – 1.7) $\rightarrow$ measured $\sim 78\text{G}$, borderline OOM.

Theory vs measured differs 1.7 $\times$: PyTorch caching allocator reservation + memory fragmentation + temporary-tensor peaks + activation underestimate. DL memory theory only estimates orders of magnitude; precise values need measurement.

Conclusion: large context + deep model must enable grad_checkpointing

grad_checkpointing: forward stores only a few checkpoint layers' activations; backward **recomputes** intermediate activations from the nearest checkpoint (discarded after use).

What's saved is "activation storage" (via recompute), **not gradients** (all computed), **not "keeping only the last layer"**.

- grad_ckpt off: all 32 layers' activations stored $\rightarrow \sim 70\text{G} \rightarrow$ OOM.
- grad_ckpt on: activations not fully stored $\rightarrow \sim 30$ – $40\text{G} \rightarrow$ even micro24 doesn't OOM.

Cost: recompute forward in backward, $\sim 25\%$ slower. **But for a deep model + long context it's a hard requirement, not optional** — turning it off forces the batch tiny yet still OOMs, a bad deal.

Precision's effect: training uses BF16 (params/grads/activations 2 bytes), only Adam states are FP32 (4 bytes, needing high-precision accumulation). If all FP32, activations double ($38 \rightarrow 76\text{G}$), OOM'ing far worse. BF16 is the key memory saver.

Reflection: 80G is actually wasteful: with grad_ckpt on, memory need is only 30–40G, so **a 40G card suffices**. Using 80G was the wrong direction of "forcing a big batch with grad_ckpt off" — not a real need.

10. Speed back-and-forth: GPU utilization, async prefetch

Symptom: GPU mostly 0%, tok/s stuck at 17k

With grad_ckpt on it ran, but `nvidia-smi` caught the GPU mostly at 0%, tok/s only 17k (14B would take ~ 9 days).

Diagnosis: synchronous data-loading stall

Testing `get_batch` alone: 2392k tok/s (data itself is blazing fast). But GPU 0% during training $\rightarrow$ **not slow data, but a synchronous stall**: GPU finishes a step, then stops to wait for the CPU

to fetch the next batch (get_batch) + pin + transfer, GPU idling. grad_accum fetches multiple times per step, more gaps.

Solution: async prefetch (pipeline overlap)

A background thread continuously fetches + pins batches into a queue; the main loop takes ready ones directly; CUDA stream transfers asynchronously. **This overlaps "CPU fetching data" with "GPU computing" (a pipeline), so the GPU doesn't idle.** It's the producer-consumer + double-buffering idea from HFT/concurrency, applied to training data loading. Effect: GPU utilization 0% → **100%**.

But — prefetch exposed the real bottleneck: GPU hit 100%, yet tok/s stayed ~18k (barely up). So data loading wasn't the main bottleneck; **grad_checkpointing's recompute is** — GPU 100% means "busy recomputing activations", effective throughput eaten by recompute. Real speedup needs less recompute: cut context to 2K (small activations, can turn grad_ckpt off, no recompute) + prefetch, reaching 40k+. This is the "run first to bank a warm start, then optimize" trade-off — stabilize with 4K+gckpt to bank a checkpoint, later switch to 2K to speed up resumed training.

Another speed trap: ckpt_every. Smoke test set `ckpt_every` to 10 and it wasn't reverted, saving an 8.1G checkpoint every 10 steps, disk IO dragging each step to 55s (only 790 steps overnight). Reverting to 100 dropped the save frequency 10× and tok/s returned to 18k. **keep_last_k=3 was working all along** (only the last 3 + best + last kept), so no disk blowup.

11. Result: steep drop and plateau

loss from 10.63 → 1.54, ppl from 41569 → 4.7 (step ~800, only 0.4B tokens trained, 3% of data). gnorm stable at 0.4-1.4 (healthy, no explosion).

Two phases of the loss curve:

- **Steep-drop phase** (early, a few hundred steps): rapidly learns basic regularities (syntax, bracket matching, common patterns) — strong, easy-to-learn regularities, loss plunges to ~1.5.
- **Plateau phase** (after): slow descent around 1.5 (grinding complex logic, long-tail patterns), diminishing returns — "1.5 lasts a long time".

So **steep-dropping to 1.5 = basic C++ is writable** (Stage A's core goal). Judge "enough or not" via generation tests (does it write plausible C++), not by waiting on loss — in the plateau loss barely moves, but generation may already be good.

Stage A summary

From scratch (random init), a hand-written Transformer pretrained on 14B C++ tokens **reaches loss 1.5 / ppl 4.7 at just ~3% of the data, writing basic C++**. Full pipeline built by hand: data cleaning (32-core parallel), hybrid tokenizer, 0.6B decoder, training loop (grad accumulation / checkpoint-resume / async prefetch). Walked through the engineering trade-offs of memory (activations/gckpt/OOM) and speed (utilization/prefetch/real bottleneck), and derived from first principles $CE = \log|\text{vocab}|$ and perplexity = effective candidate count.

— End of Stage 1. Next: B (SFT problem-solution alignment) + C (RLVR reinforcing real solving). —